Linux Gstreamer
开发指南

版本号: 1.1
发布日期: 2024.9.20

广州星翼电子科技有限公司zdyzlinux

版本历史

版本号	日期	制/修订人	内容描述
1.0	2024.3.8	AWA2127	初始版本
1.1	2024.9.20	AWA2127	添加 Buildroot 配置

广州星翼电子科技有限公司zdyzlinux

版权所有 © 珠海全志科技股份有限公司。保留一切权利

目　录

1 前言 **1**
1.1 文档简介 1
1.2 目标读者 1
1.3 适用范围 1

2 Gstreamer **2**
2.1 Gstreamer 简介 2
2.2 Gstreamer 视频解码适配方案 3

3 gst-omx 介绍 **4**
3.1 gst-omx 配置 4
3.2 gst-omx 性能优化 4

4 gst-omx 对接 **7**
4.1 Gstreamer 原生资源 7
4.2 gst-omx 编译 8
4.2.1 源码获取 8
4.2.2 Buildroot 编译 8
4.2.3 源码编译 8
4.2.4 验证测试 9

5 常用命令 **10**

6 常用插件说明 **11**
6.1 Source 11
6.2 Sink 12

7 模块使用范例 **13**
7.1 命令行使用说明 13
7.1.1 播放视频 13
7.1.2 保存 YUV 数据 13
7.1.3 获取视频信息 13
7.2 VLC 对接说明 14
7.3 播放器开发 15

8 FAQ **16**
8.1 [日志功能] 16
8.2 [gst-inspect 无法找到 omx 插件] 16
8.3 [播放视频绿边] 16
8.4 [播放视频卡顿] 17

版权所有 © 珠海全志科技股份有限公司。保留一切权利

1 前言

1.1 文档简介

本文档主要介绍 Gstreamer 及相关插件的编译和使用方法。

1.2 目标读者

多媒体软件开发人员

1.3 适用范围

表 1-1: 适用产品列表

产品名称	内核版本	操作系统
T527 Linux AIOT	Linux-5.15	Ubuntu
T536	Linux-5.10	Buildroot

版权所有 © 珠海全志科技股份有限公司。保留一切权利

2 Gstreamer

2.1 Gstreamer 简介

GStreamer 是一个开源的多媒体框架库。利用它，可以构建一系列的媒体处理模块，包括从简单的 ogg 播放功能到复杂的音频（混音）和视频（非线性编辑）的处理。Gstreamer 并不受限于音频和视频的处理，它能够处理任意类型的数据流。Gstreamer 架构如下图所示：

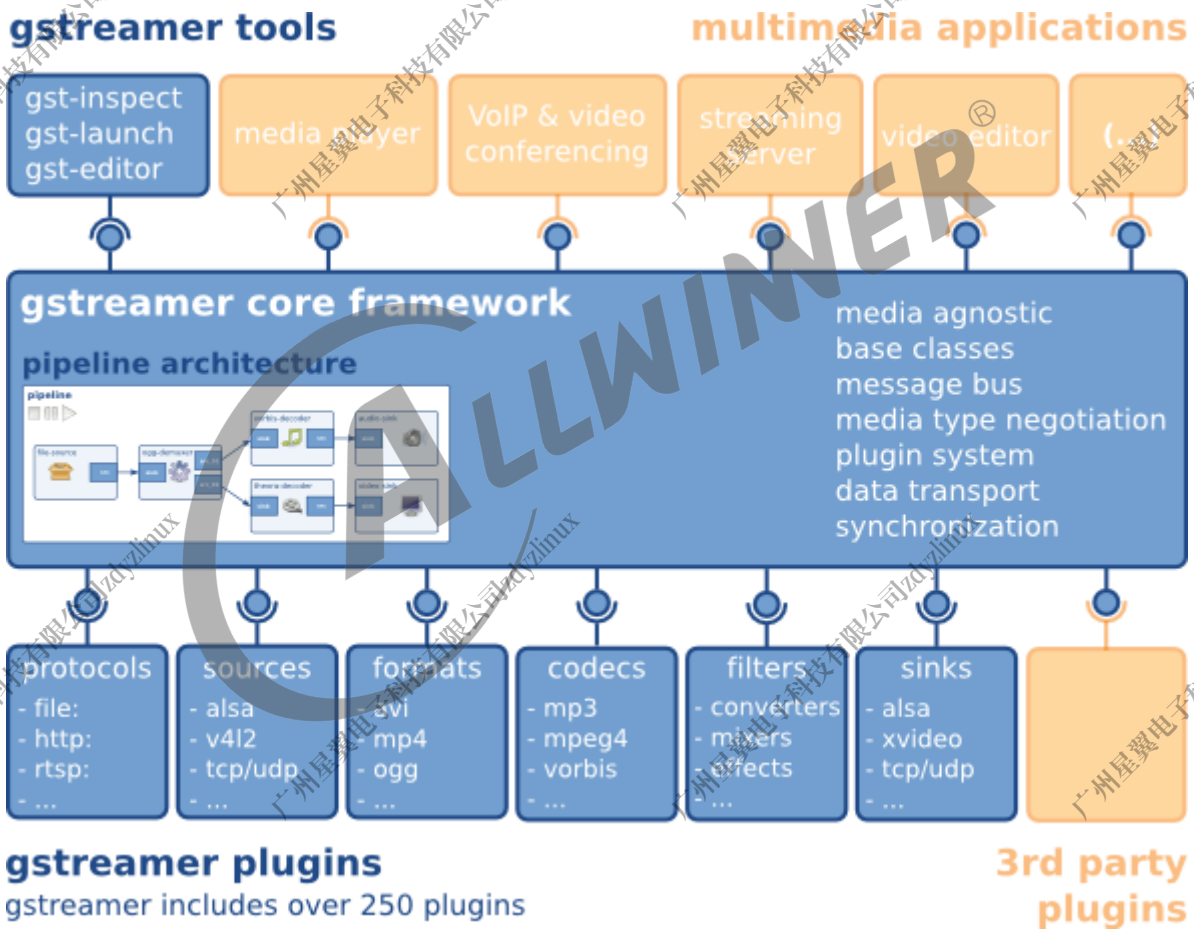

图 2-1

GStreamer 框架是基于插件的, 有些插件中提供了各种各样的多媒体数字信号编解码器, 也有些提供了其他的功能。所有的插件都能够被链接到任意的已经定义了的数据流管道中。

应用程序可以透明的利用解码和过滤技术。开发者可以使用简洁通用的接口来编写一个简单的插件来添加新的解码器或滤镜。目前，gstreamer 已经可以应用在 Linux、windows、macos、an-

版权所有 © 珠海全志科技股份有限公司。保留一切权利

droid、unix 等多个平台上了，而且有全世界的开源工程师们在使用和维护。基于 gstreamer 的易用性，gstreamer 得到了极大的推广。

2.2 Gstreamer 视频解码适配方案

本地视频播放流程如下：

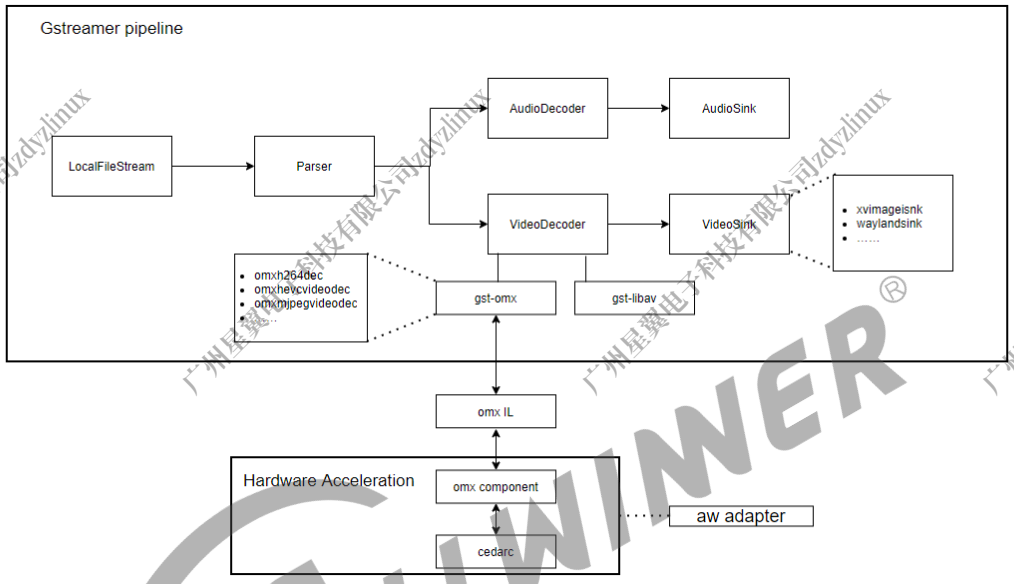

图 2-2

视频文件经过 parser 解析出音视频数据，分别送到对应的解码器中。视频流经过 VideoDecoder 解码，最后通过显示插件（xvimagesink）进行显示。

Gstreamer 基于插件的特性，所以当我们要实现解码功能时，只需要关注解码插件，其余插件使用 Gstreamer 原生提供的即可。Gstreamer 原生也提供了一些软解方案（如 gst-libav，基于 ffmpeg 实现），也可以通过一些标准的框架（如 gst-omx，基于 openmax）对接到厂商的硬件解码。

全志平台 Gstreamer 视频解码适配方案：通过 gst-omx 插件来实现视频解码的硬件加速。gst-omx 插件是基于 openmax 实现的，主要是通过 openmax IL 层的接口，对接 Gstreamer 的解码流程。这部分内容 Gstreamer 原生已经实现，厂商只需要实现 openmax 定义的接口，并通过配置文件使能对应的组件即可。

目前已支持的解码插件：h264、h265、h263、mjpeg、mpeg1/2/4、vp9

说明

此处的格式为插件支持，对应格式的解码是否可用需查阅对应的芯片规格

版权所有 © 珠海全志科技股份有限公司。保留一切权利

3 gst-omx 介绍

gst-omx 插件是 GStreamer 多媒体框架中的一个插件，用于与 OpenMAX IL（Integration Layer）标准兼容的硬件解码器进行交互。是基于 Gstreamer 原有的 GstVideoDecoder 类实现的子类 gstomxvideodec。

3.1 gst-omx 配置

gst-omx 是 Gstreamer 原生提供的，厂商实现自己的 openmax 组件后，需要通过配置文件，将组件对接到 gst-omx，完成解码插件的注册。
gst-omx 在初始化时，会读取 gstomx.conf 文件，该文件一般放在机器端的/etc/xdg 目录下，该配置文件主要是说明 element 的名称、rank、port 等信息。以 h264 解码为例：

```
[omxh264dec]     //解码插件名
type-name=GstOMXH264Dec     //解码器类型，需要与gstomxh264dec.h中定义的类一致
core-name=/usr/lib/aarch64-linux-gnu/libOmxCore.so    //openmax核心库路径
component-name=OMX.allwinner.video.decoder.avc     //openmax组件名
rank=257       //解码插件使用优先级，rank值越高，优先级越高
in-port-index=0  //openmax组件输入端口的索引
out-port-index=1 //openmax组件输出端口的索引
hacks=event-port-settings-changed-ndata-parameter-swap;event-port-settings-changed-port-0-to-1;no-disable-outport
```

通过以上内容，我们就声明了一个 h264 解码器的插件。

3.2 gst-omx 性能优化

在原有的软件通路中，存在以下的拷贝：

版权所有 © 珠海全志科技股份有限公司。保留一切权利

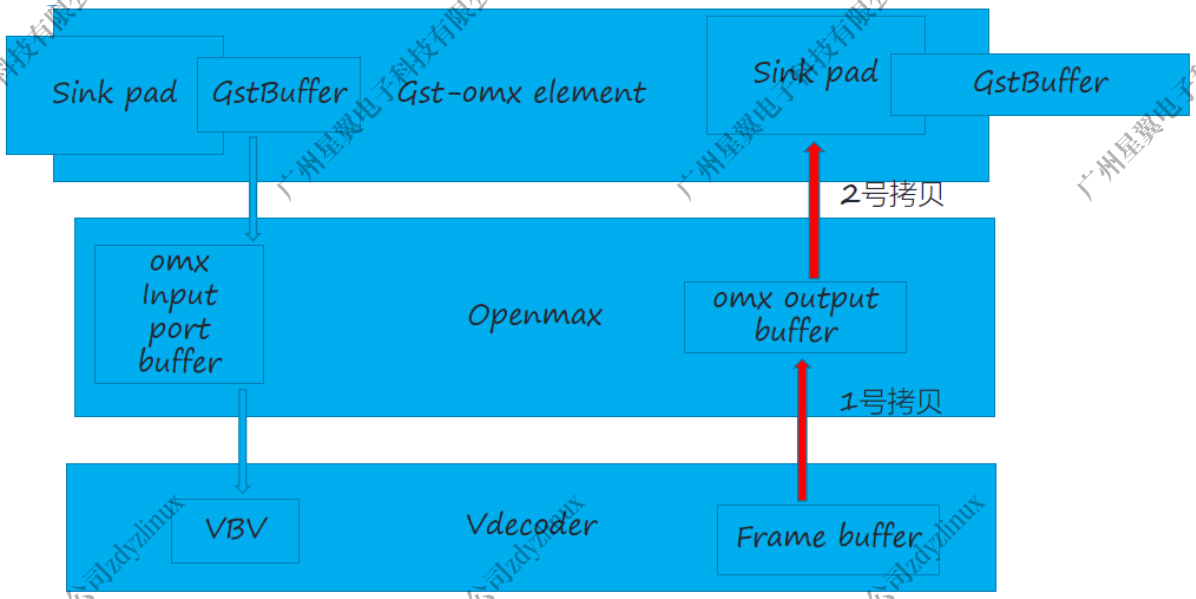

图 3-1

输入端的拷贝由于数据量不大，所以对性能没有什么影响，我们需要优化的是输出通路的拷贝，优化方法如下：

1 号拷贝优化由 openmax 实现修改，确保 alloc buffer 时为 ion 内存，并通过外部设置的方法设置到 vdecoder 中。

2 号拷贝优化方法：使用 GstOmxBufferPool 来实现映射式的零拷贝；

在实现 2 号拷贝优化时，我们需要分清 GstBuffer 和 GstOMXBuffer 之间的关系：

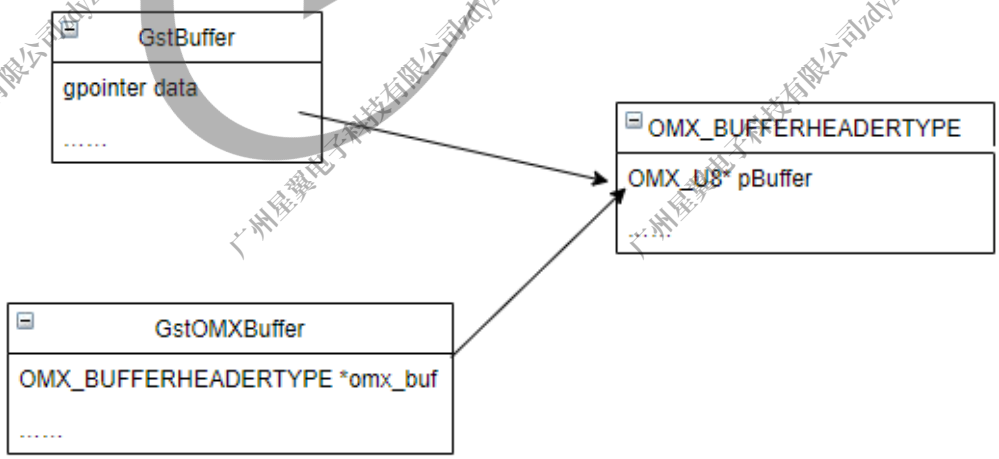

图 3-2

GstOMXBuffer：gstomxvideodec 与 omx 之间交互使用的 Buffer。

GstBuffer：插件之间传递的 Buffer。

版权所有 © 珠海全志科技股份有限公司。保留一切权利

在解码插件中，实际上存在两个 BufferPool，一个用于存放 GstOMXBuffer，一个用于存放 GstBuffer，GstOMXBuffer 和 GstBuffer 中实际的数据是同一个 pBuffer。

在实现 1 号拷贝时，我们通过一个私有数据来描述所使用的 buffer，即 pBuffer 指向的是私有数据，而非实际的图像数据。但原生的 gst-omx 实现中，传递给下游插件的 GstBuffer 是指向实际图像数据的，为保持对原生框架的兼容，有以下两种方法：

1. 在创建 GstBuffer 队列时，将 GstBuffer 的数据指针指向实际的图像数据；
2. 在获取到 GstOMXBuffer 后，从私有数据中提取出虚拟地址，构建一个新的 GstBuffer 对象，传给下游插件。

目前采用的是第二种方法，通过 wrapper_omx_buffer 函数实现。

版权所有 © 珠海全志科技股份有限公司。保留一切权利

4 gst-omx 对接

由于 gstreamer 中各个插件都是独立的，所以对接过程中，我们只需要在原有的视频通路中，将解码插件替换为 gst-omx 即可。gst-omx 的对接流程主要分为两步：

1. 安装 Gstreamer 原生的插件，保证视频通路可以正常跑通，可以使用软解方案进行验证。
2. 下载 gst-omx 源码，打上补丁，编译

4.1 Gstreamer 原生资源

Gstreamer 原生提供的模块主要有以下几个包

gst-plugins-base: gstreamer 应用所需的必要插件。
gst-plugins-good: 高质量的采用 LGPL 授权的插件。
gst-plugins-ugly: 高质量，但使用了 GPL 等其他授权方式的库的插件，比如使用 GPL 的 x264，x265。
gst-plugins-bad: 质量有待提高的插件，成熟后可以移到 good 插件列表中。
gst-libav: 对 libav 封装，使其能在 gstreamer 框架中使用。

可以通过以下命令安装 gstreamer 原生插件：

```
sudo apt-get install libgstreamer1.0-dev
sudo apt-get install libgstreamer-plugins-base1.0-dev
sudo apt-get install libgstreamer-plugins-good1.0-dev
sudo apt-get install libgstreamer-plugins-bad1.0-dev
sudo apt-get install gstreamer1.0-libav
sudo apt-get install libgstreamer-plugins-ugly1.0-dev
```

安装后，可以通过以下命令验证：

```
# 使用ffmpeg的h264进行解码
gst-launch-1.0 filesrc location=720_1.mp4 ! qtdemux ! avdec_h264 ! autovideoconvert ! xvimagesink

# 产生一个测试视频
gst-launch-1.0 -v videotestsrc pattern=ball name=left ! autovideosink
```

说明

如果要 decodebin 或者 gst-play 可以选择 ffmpeg 进行解码，可以在 gstomx.conf 中将对应的解码器删除，或者降低优先级，ffmpeg 解码的默认 rank 值为 256。

版权所有 © 珠海全志科技股份有限公司。保留一切权利

4.2 gst-omx 编译

4.2.1 源码获取

Ubuntu

gst-omx 源码可以通过 github 或者 https://gstreamer.freedesktop.org/src/gst-omx 下载，然后打上由全志提供的补丁。目前主要提供 1.18 和 1.20.1 分支的补丁。

Buildroot

gst-omx 插件补丁在 buildroot/buildroot-202205/package/gstreamer1/gst-omx 目前仅提供 1.20.1 分支的补丁，原生的插件源码均可通过网络下载。

4.2.2 Buildroot 编译

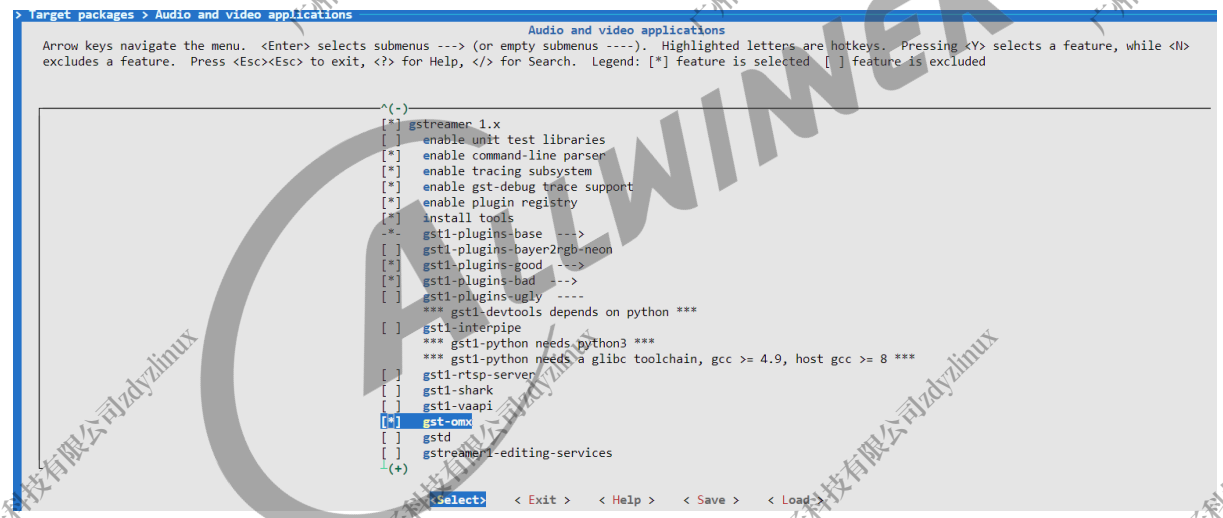

图 4-1

在 buildroot/buildroot-202205/目录下执行 make menuconfig 后打开 Gstreamer 相关配置即可，若对应的插件缺少源码，可自行通过网络下载后放到 SDK 里。

4.2.3 源码编译

安装 meson、ninja：

```
sudo apt-get update
sudo apt-get install python3-pip ninja-build
pip3 install meson
```

版权所有 © 珠海全志科技股份有限公司。保留一切权利

说明

需要先修改/etc/apt/source.list 文件，将 dep-src 的注释去掉，否则会出现找不到源的问题。

下载好源码，打上补丁后，进入 gst-omx 的根目录

```
root@localhost:/home# cd gst-omx/
root@localhost:/home/gst-omx# meson build -Dstruct_packing=8 -Dtarget=cedarx --prefix=/usr && ninja -C build install

struct_packing：指定结构体对齐方式为64字节对齐
target：指定使用哪个平台的OMX，在安装时会根据target安装对应的gstomx.conf
```

编译完后，可以通过 gst-inspect-1.0 查看是否安装成功

```
root@localhost:/home/gst-omx# gst-inspect-1.0 |grep omx
omx:  omxavsvideodec: OpenMAX AVS Video Decoder
omx:  omxdivx4videodec: OpenMAX DIVX4 Video Decoder
omx:  omxdivxvideodec: OpenMAX divx Video Decoder
omx:  omxh263videodec: OpenMAX H.263 Video Decoder
omx:  omxh264dec: OpenMAX H.264 Video Decoder
omx:  omxh264videoenc: OpenMAX H.264 Video Encoder
omx:  omxhevcvideodec: OpenMAX H.265 Video Decoder
omx:  omxmjpegvideodec: OpenMAX MJPEG Video Decoder
……
```

4.2.4 验证测试

编译后，可以通过以下命令验证解码是否正常：

```
#播放视频
gst-launch-1.0 filesrc location=720_1.mp4 ! qtdemux ! omxh264dec ! autovideoconvert ! xvimagesink

#dump解码图像
gst-launch-1.0 filesrc location=720_1.mp4 ! qtdemux ! omxh264dec ! filesink location=/tmp/h264.yuv
```

版权所有 © 珠海全志科技股份有限公司。保留一切权利

5 常用命令

- gst-lauch-1.0
  Gstreamer 启动器，用来手动连接不同的 element，通过! 连接两个 element。用法如下：

```
gst-launch-1.0 videotestsrc pattern=5 ! xvimagesink
```

- gst-play-1.0
  Gstreamer 播放器，用于播放各种音视频文件、流媒体。

```
# 使用默认videosink
gst-play-1.0 /tmp/test.mp4

# 指定xvimagesink进行显示
gst-play-1.0 /tmp/test.mp4 --videosink=xvimagesink
```

- gst-inspect-1.0
  查找器，用于列出所有插件或某一插件的详细信息。

```
#列出所有插件，不带任何参数
gst-inspect-1.0

#查看某一插件的详细信息，参数为具体某个插件的名字
gst-inspect-1.0 omxh264dec
```

- gst-discoverer-1.0
  GStreamer 的一个工具库，用于分析和检测媒体文件的属性、元数据和流信息。可以地获取媒体文件的各种信息，如时长、编解码器、分辨率、音频格式等。

```
gst-discoverer-1.0 /tmp/MJPEG.mp4
```

版权所有 © 珠海全志科技股份有限公司。保留一切权利

6 常用插件说明

6.1 Source

Source 插件指数据源，即可以产生数据但不能接收数据，常用的 Source 插件有以下几个：

- filesrc

gstreamer 的核心插件之一，用于从文件系统中读取某一文件。

```
gst-launch-1.0 filesrc location=/tmp/test.mp4 ! filesink location=/tmp/test.mp4
```

- videotestsrc

gstreamer 自带的测试视频源，可通过 parttern 属性选择不同的图案。

```
通过命令行可以查询可用的pattern：
gst-inspect videotestsrc
……
  pattern          : Type of test pattern to generate
                     flags: readable, writable
                     Enum "GstVideoTestSrcPattern" Default: 0, "smpte"
                       (0): smpte          - SMPTE 100% color bars
                       (1): snow           - Random (television snow)
                       (2): black          - 100% Black
                       (3): white          - 100% White
                       (4): red            - Red
                       (5): green          - Green
                       (6): blue           - Blue
                       (7): checkers-1     - Checkers 1px
                       (8): checkers-2     - Checkers 2px
                       (9): checkers-4     - Checkers 4px
                       (10): checkers-8    - Checkers 8px
                       (11): circular      - Circular
                       (12): blink         - Blink
                       (13): smpte75       - SMPTE 75% color bars
                       (14): zone-plate    - Zone plate
                       (15): gamut         - Gamut checkers
                       (16): chroma-zone-plate - Chroma zone plate
                       (17): solid-color   - Solid color
                       (18): ball          - Moving ball
……
# 使用默认格式输出视频
gst-launch-1.0 videotestsrc pattern=5 ! xvimagesink
# 使用指定格式输出视频
gst-launch-1.0 videotestsrc pattern=3 ! "video/x-raw,width=1920,height=1080,format=(string)YV12" ! xvimagesink
```

版权所有 © 珠海全志科技股份有限公司。保留一切权利

6.2 Sink

Sink 插件指可以接受数据但不会发送数据的插件。

- filesink

用于存放数据到文件中，用法如下：

```
gst-launch-1.0 filesrc location=/tmp/test.mp4 ! filesink location=/tmp/test1.mp4
```

- fakesink

万能的结束元件，可以接受任何格式的数据，但不做任何处理，常用于调试过程中结束一条流水线，用法如下：

```
gst-launch-1.0 filesrc location=/tmp/test.mp4 ! fakesink
```

- xvimagesink

视频 Sink，接受图像数据并显示，基于 X11 接口实现，用法如下：

```
gst-launch-1.0 videotestsrc ! xvimagesink
```

版权所有 © 珠海全志科技股份有限公司。保留一切权利

7 模块使用范例

7.1 命令行使用说明

7.1.1 播放视频

```
gst-play-1.0 /tmp/test.mp4
gst-launch-1.0 filesrc location=/tmp/test.mp4 ! parsebin ! omxh264dec ! xvimagesink
```

7.1.2 保存 YUV 数据

```
gst-launch-1.0 filesrc location=/tmp/h264.mp4 ! parsebin ! omxh264dec ! filesink location=/tmp/h264.yuv
```

7.1.3 获取视频信息

```
root@localhost:/# gst-discoverer-1.0 /tmp/h264.mp4
Analyzing file:///tmp/h264.mp4
Done discovering file:///tmp/h264.mp4
Properties:
  Duration: 0:00:04.871533333
  Seekable: yes
  Live: no
  container #0: Quicktime
    audio #1: MPEG-4 AAC
      Stream ID: (null)
      Language: <unknown>
      Channels: 2 (front-left, front-right)
      Sample rate: 48000
      Depth: 0
      Bitrate: 0
      Max bitrate: 0
    video #2: H.264 (High Profile)
      Stream ID: f936bf1f618e89178242f705ec9412754753e6760094e81a1597ac29f4309e64/001
      Width: 1920
      Height: 1088
      Depth: 24
      Frame rate: 60000/1001
      Pixel aspect ratio: 1/1
      Interlaced: false
      Bitrate: 24006410
      Max bitrate: 0

若gst-discoverer-1.0返回Missing plugins，说明缺少相应的插件。
```

版权所有 © 珠海全志科技股份有限公司。保留一切权利

```
root@localhost:/# gst-discoverer-1.0 /tmp/MJPEG.mp4
Analyzing file:///tmp/MJPEG.mp4
Done discovering file:///tmp/MJPEG.mp4
Missing plugins
 (gstreamer|1.0|gst-discoverer-1.0|MPEG-4 Video (Simple Profile) decoder|decoder-video/mpeg, mpegversion=(int)4,
      systemstream=(boolean)false, profile=(string)simple, level=(string)3)
 (gstreamer|1.0|gst-discoverer-1.0|MPEG-4 AAC decoder|decoder-audio/mpeg, mpegversion=(int)4, level=(string)2, base-
      profile=(string)lc, profile=(string)lc)
```

7.2 VLC 对接说明

VLC 源码中已经有对 Gstreamer 解码的支持，具体实现在项目源码的 modules/codec/gstreamer/目录下。所以对接 VLC 只需要以下几步：

1. 安装 Gstreamer、VLC 依赖；
2. 下载源码；
3. VLC 编译配置，使能 Gstreamer，编译；
4. 运行 VLC，在图像化界面中进行设置。

- 下载依赖

```
sudo apt-get update
sudo apt-get build-dep vlc
sudo apt-get install git g++ make libtool automake autopoint pkg-config flex bison lua5.2
sudo apt install libxcb-xkb-dev
sudo apt-get install libgstreamer1.0-dev
sudo apt-get install libgstreamer-plugins-base1.0-dev
```

说明

需要先修改/etc/apt/source.list 文件，将 dep-src 的注释去掉，否则会出现找不到源的问题。

- 源码编译

  github 上下载最新版本 vlc：https://github.com/videolan/vlc

```
git clone https://github.com/videolan/vlc.git  //需要切换到3.x分支上，master分支依赖高版本QT
cd vlc
./bootstrap
./configure --enable-omxil --disable-vdpau --enable-run-as-root --enable-omxil-vout --enable-ogg  --enable-gstreamer
make -j8 && make install
```

- 播放器设置

1. 输入 vlc，运行 vlc 播放器；
2. ctrl + p 打开设置界面；

版权所有 © 珠海全志科技股份有限公司。保留一切权利

3. 点击左下角 All -> 点击 Video codecs -> 在 Preferred docders list 一栏输入 GstDecode -> 点击 save；
4. 播放视频。

说明

若运行 vlc 时找不到 libvlc.so，可以运行 export LD_LIBRARY_PATH=/usr/local/lib/ 或在 /etc/ld.so.conf.d/aarch64-linux-gnu.conf 中加入/usr/local/lib/

7.3 播放器开发

实现 gstreamer 播放器，可以参考 gst-play 的实现，其源码在 gst-plugin-base 的仓库中查找。
gst-plugin-base 仓库路径：https://gitlab.freedesktop.org/gstreamer/gst-plugins-base
gst-play 代码位置：tools/gst-play.c

版权所有 © 珠海全志科技股份有限公司。保留一切权利

8 FAQ

8.1 [日志功能]

Gstreamer 日志等级分为 ERROR(1)，WARNING(2)，FIXME(3)，INFO(4)，DEBUG(5)，LOG(6)，TRACE(7) 等。

```
#设置环境变量
export GST_DEBUG=4

#在命令行前指定，命令结束即失效
GST_DEBUG=2 gst-launch-1.0 ...

#在命令行后指定，命令结束即失效
gst-launch-1.0 .... --gst-debug=5

#指定不同模块不同日志等级
GST_DEBUG=xvimagesink:5,gstvideodecoder:3
```

8.2 [gst-inspect 无法找到 omx 插件]

问题现象：gst-inspect 无法找到 omx 的插件，报错 No such element or plugin ‘omxh264dec’

问题分析：可能导致该问题的原因有以下几个：
1. libgstomx.so 不存在，或位置错误；
2. gstomx.conf 没有配置，或配置错误；
3. gst-omx 流程出问题，导致注册失败。

问题排查步骤
1. 确认 libgstomx.so 是否在/usr/lib/aarch64-linux-gnu/gstreamer-1.0/路径，若不在，可以使用 GST_PLUGINS_PATH 变量指定 libgstomx.so 的路径；
2. 确认 gstomx.conf 中的配置是否正常，可参考 3.1 章节；
3. 打开日志功能，运行 gst-inspect-1.0 omxh264dec 查看是否报错。

8.3 [播放视频绿边]

问题现象：播放 720p 视频出现绿边

版权所有 © 珠海全志科技股份有限公司。保留一切权利

问题分析：播放出现绿边一般是由于硬件解码时用到的 buffer 需要进行对齐，而显示时使用的宽高大于实际宽高导致的。

问题排查步骤：

1. 使用 export GST_DEBUG=“xvimagesink:5” 打开 xvimagesink 的 log;
2. 使用 gst-play-1.0 播放视频，抓取 log;
3. 从 log 中定位 gst_xv_image_sink_setcaps 函数的 log，查看宽高信息;
4. 通过 ffprobe 查看对应视频的宽高信息;

对比步骤 3、4 中获取的宽高，发现 xvimagesink 设置的宽高为 736x480，而 ffprobe 获取的宽高为 720x480。由此可以确定本问题是由于对齐问题导致的。

解决方案：

- gst-play-1.0 播放：使用 GstVideoCropMeta 将实际宽高信息传递给 xvimagesink。具体实现在 wrapper_omx_buffer 函数中。
- vlc 播放：vlc 的显示插件中没有对 GstVideoCropMeta 处理，但是可以通过 i_visible_width 指定对应的 crop。修改 modules/codec/gstreamer/gstdecode.c 代码，在 DecodeBlock 函数中，加入以下修改：

```
@@ -781,6 +787,7 @@ static int DecodeBlock( decoder_t *p_dec, block_t *p_block )
     else
     {
        GstVideoFrame frame;
+       GstVideoCropMeta *crop;

        /* Get a new picture */
        if( decoder_UpdateVideoFormat( p_dec ) )
           return VLCDEC_ECRITICAL;
        }

+       crop = gst_buffer_get_video_crop_meta(p_buf);
+       if (crop)
+        {
+          p_pic->format.i_x_offset = crop->x;
+          p_pic->format.i_y_offset = crop->y;
+          p_pic->format.i_visible_width = crop->width;
+          p_pic->format.i_visible_height = crop->height;
+       }
```

8.4 [播放视频卡顿]

问题现象：gst-play-1.0 播放 1080P 视频卡顿，log 打印 “WARNING A lot of buffers are being dropped”

问题分析：“WARNING A lot of buffers are being dropped” 说明此时发生了丢帧。此时有两种可能：

1. 显示渲染太慢，可以通过–videosink=xvimagesink，指定 xvimagesink 进行播放。

版权所有 © 珠海全志科技股份有限公司。保留一切权利

2. 解码耗时太久，需要进一步从解码流程中分析。

问题排查步骤：

1. 若使用 xvimagesink 播放正常，则说明卡顿可能是由于显示渲染没有使用硬件加速;
2. 使用以下命令，将解码后的数据送到 fakesink，若时间进度条明显很慢，则说明解码流程中出现问题

```
gst-launch-1.0 filesrc location=/tmp/MJPEG.mp4 ! parsebin ! decodebin ! fakesink
```

针对步骤 2 出现的解码流程问题，可通过以下方法排查：

1. 在 gst_omx_video_dec_handle_frame 函数中检查输入端数据是否送慢了;
2. 在 gst_omx_video_dec_loop 检查获取输出数据是否拿慢了。

通过排查解码流程，定位到是输入端出现问题，inputBuffer 送到 omx 组件后，返回很慢。原因是在 omx 组件的实现中，根据收到的 packet 的数量去判断是否延缓送数据给 sbm，导致没有将 inputBuffer 返回给应用。

解决方案

修改 omx 组件的逻辑，判断延缓送数据的条件改为使用 sbm 队列中的 bufferSize 去判断。

版权所有 © 珠海全志科技股份有限公司。保留一切权利

著作权声明

版权所有 ©2024 珠海全志科技股份有限公司。保留一切权利。

本文档及内容受著作权法保护，其著作权由珠海全志科技股份有限公司（“全志”）拥有并保留一切权利。

本文档是全志的原创作品和版权财产，未经全志书面许可，任何单位和个人不得擅自摘抄、复制、修改、发表或传播本文档内容的部分或全部，且不得以任何形式传播。

商标声明

ALLWINNER、ALLWINNER 全志科技、全志科技、ALLWINNER（不完全列举）均为珠海全志科技股份有限公司的商标或者注册商标。在本文档描述的产品中出现的其它商标，产品名称，和服务名称，均由其各自所有人拥有。

免责声明

您购买的产品、服务或特性应受您与珠海全志科技股份有限公司（“全志”）之间签署的商业合同和条款的约束。本文档中描述的全部或部分产品、服务或特性可能不在您所购买或使用的范围内。使用前请认真阅读合同条款和相关说明，并严格遵循本文档的使用说明。您将自行承担任何不当使用行为（包括但不限于如超压，超频，超温使用）造成的不利后果，全志概不负责。

本文档作为使用指导仅供参考。由于产品版本升级或其他原因，本文档内容有可能修改，如有变更，恕不另行通知。全志尽全力在本文档中提供准确的信息，但并不确保内容完全没有错误，因使用本文档而发生损害（包括但不限于间接的、偶然的、特殊的损失）或发生侵犯第三方权利事件，全志概不负责。本文档中的所有陈述、信息和建议并不构成任何明示或暗示的保证或承诺。

本文档未以明示或暗示或其他方式授予全志的任何专利或知识产权。在您实施方案或使用产品的过程中，可能需要获得第三方的权利许可。请您自行向第三方权利人获取相关的许可。全志不承担也不代为支付任何关于获取第三方许可的许可费或版税（专利税）。全志不对您所使用的第三方许可技术做出任何保证、赔偿或承担其他义务。